

Rally Supply Project TA 2 Documentation

Required database Models and the expanded pickleball point of sale

This guide describes the separate pickleball-pos-ta2 project. The two required assessment tables remain exactly as supplied. New POS features use additional tables, Models, controllers, views and local setup helpers. All demo people and products are fictional.

Assessment requirements

TA2 requirement	Implemented in
MySQLi connection	.env and app/Config/Database.php
Required tables and five records each	database/schema.sql and sample-data.sql
CustomerModel and UserModel	app/Models/CustomerModel.php and UserModel.php
Model and Query Builder retrieval	Customers::index and Users::index call findAll()
Views display database records	app/Views/accounts/customers.php and users.php

Expanded features

A database catalog has nine products and 29 stocked options. A shopper can place a pending website order without staff sign-in. Staff can review orders, update inventory, and record separate in-person sales with receipts. Customer and user pages also support name search and allowed sort choices.

Local runtime

Verified with CodeIgniter 4.7.4, PHP 8.5.11 and XAMPP MariaDB 10.4.32 through MySQLi. The local database listens at 127.0.0.1:3307 and the website at localhost:8082. Asia/Manila is the application time zone. MariaDB is MySQL-compatible; Oracle MySQL was not separately tested.

Required TA2 database schema

These columns and constraints reproduce the instructor's SQL. The optional phone remains nullable. No password, role, inventory or sales field was added to either required table.

database/schema.sql

```
-- TA2 required schema. Run once in a new database.
-- No DROP commands: an existing table produces an error rather than losing data.
CREATE DATABASE IF NOT EXISTS rally_supply_ta2
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE rally_supply_ta2;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone VARCHAR(20),
  created_at DATETIME NOT NULL
);

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  created_at DATETIME NOT NULL
);
```

Sample account records

Customers	Users
Mika Reyes	courtadmin · Alex Rivera
Andre Santos	rallycashier · Jamie Lim
Bianca Cruz	paddletech · Morgan Lee
Paolo Mendoza	servecrew · Sam Dela Cruz
Toni Garcia	matchdesk · Taylor Navarro

The complete INSERT statements, email addresses, phone numbers and creation dates are in database/sample-data.sql. Import that file only once into new TA2 tables; the launcher never resets existing account rows.

Optional POS database extension

The POS tables are separate from customers and users. Product variants hold stock for each grip size, color or apparel size. Staff-recorded sales keep the staff and optional customer ID. Website orders keep shopper contact information separately and require no staff login.

Table	Key columns and relationship
products	id; unique sku; name, category, price_cents, photo, specs
product_variants	id; product_id → products.id; unique sku; stock_qty
staff_credentials	user_id → users.id; password_hash; is_active
sales	id; customer_id → customers.id; user_id → users.id; total_cents
sale_items	id; sale_id → sales.id; variant_id → product_variants.id; item snapshot
web_orders	id; unique private token; buyer contact, status and total
web_order_items	id; order_id → web_orders.id; item and price snapshot

Why separate tables?

The required users table has no password column, so authentication data belongs in staff_credentials. A sale can include multiple items, so sale_items is separate from sales. Product stock belongs to product_variants because a shoe size or paddle grip can run out while other options remain available.

Money and stock

Selected schema fields

```
price_cents INT UNSIGNED NOT NULL
stock_qty INT UNSIGNED NOT NULL DEFAULT 0
-- Example: 489000 centavos = ₱4,890.00
```

Whole centavos avoid floating point rounding in totals. database/commerce-schema.sql creates the extension; commerce-seed.sql provides nine products and 29 variants. Repeated launcher runs preserve edits, stock, orders and sales.

Getting started

Automatic setup on this Windows machine

1. Open the extracted pickleball-pos-ta2 folder in VS Code or File Explorer.
2. Double-click Start-TA2.cmd and keep its terminal open.
3. Open <http://localhost:8082/>, browse Shop, and place an order from Cart without signing in.
4. Open .local/staff-login.txt for the generated first username and password.
5. Sign in only for Customers, Users, Inventory, Orders and in-person Sales.
6. Press Ctrl+C to stop the website; run Stop-Database.cmd to stop the database.

The launcher checks that port 3307 belongs to this project before importing anything. On a fresh copy it creates the TA2 database, the extension and a private random staff password. Later launches preserve account rows, product stock, website orders, sales and credentials. The .env and .local folders are excluded from the ZIP and future Git repository.

Manual MySQL or XAMPP setup

Start MySQL in XAMPP. Import schema.sql and sample-data.sql once, then commerce-schema.sql and commerce-seed.sql. Copy .env.example to .env and enter the real connection settings. Install Composer dependencies if vendor is absent. Run the command below to create the first staff password, then serve the site manually.

Run from the project folder with mysqli enabled

```
php spark staff:password
php spark serve --host localhost --port 8082
```

The first staff password is written to .local/staff-login.txt. For another existing user, run `php spark staff:password USERNAME`. That command creates or resets the user's password and writes a separate private file in .local. Start-TA2.cmd manages its own port-3307 instance and rejects a manual .env pointed at a different port.

Account Models, queries and views

The required account Models retain id as primary key and return arrays. Their allowedFields map the exact required columns. Automatic timestamps are off because the supplied tables have no updated_at.

Key mapping from app/Models/CustomerModel.php

```
class CustomerModel extends Model
{
    protected $table = 'customers';
    protected $primaryKey = 'id';
    protected $returnType = 'array';
    protected $allowedFields = ['full_name', 'email', 'phone', 'created_at'];
    protected $useTimestamps = false;
}
```

Customer retrieval in Customers::index

```
$totalCustomers = $model->countAllResults();
if ($search !== '') {
    $model->like('full_name', $search);
}
$customers = $model
    ->orderBy($sortOptions[$sort][1], $sortOptions[$sort][2])
    ->findAll();
```

Users::index uses the same pattern with a grouped name or username search. Sort values are limited to a fixed list before a column name reaches Query Builder. The views escape database content, show a separate no-match state, and display friendly dates while keeping the exact DATETIME in the database.

Account view excerpt

```
<td><?= esc($customer['full_name']) ?></td>
<td><?= esc($customer['phone']) ?: 'Not provided' ?></td>
<time datetime="<?= esc(str_replace(' ', 'T',
    $customer['created_at']), 'attr') ?>">
    <?= esc(date('M j, Y', strtotime($customer['created_at']))) ?>
</time>
```

Catalog, cart and inventory

Catalog::index reads active products through ProductModel, obtains their active variants through ProductVariantModel, and groups the variants under each product. Category filters select a known category key. Prices, stock and product descriptions come from the database.

Catalog query pattern

```
$products = $model->orderBy('id', 'ASC')->findAll();
$variants = (new ProductVariantModel())
    ->whereIn('product_id', array_column($products, 'id'))
    ->where('is_active', 1)->findAll();
```

A product card displays photo, description, three comparison facts, variant stock and an Add to cart form. CartService stores variant IDs and quantities in the session, checks stock before adding or updating an item, and calculates the current cart total from product records.

Cart stock guard

```
if ($newQuantity > (int) $variant['stock_qty']) {
    return 'There is not enough stock for that option.';
}
$cart[$variantId] = $newQuantity;
session()->set('cart', $cart);
```

Inventory::index shows all 29 variants with their current quantities. An authorized staff member can save a nonnegative whole number from 0 to 9999 for one variant. Cart quantities are checked again at checkout, so a stock adjustment cannot silently create a negative sale.

Staff sign-in and in-person sale

The StaffOnly filter protects account rosters, inventory, website order management, in-person sale checkout, the sales list and receipts. Shopper checkout is public. Passwords are hashed in staff_credentials, and successful sign-in regenerates the session ID. All POST forms contain CodeIgniter CSRF fields. The first local password is generated randomly and saved only in the private .local folder.

Sign-in pattern from Auth::login

```
$valid = $credentials !== null
    && (int) $credentials['is_active'] === 1
    && password_verify($password, $credentials['password_hash']);
if ($valid) {
    $session->regenerate(true);
    $session->set(['staff_id' => (int) $user['id']]);
}
```

Sales::store remains available to signed-in staff at /pos/checkout. It validates a selected customer, computes the total from server-side cart items, then starts one database transaction. It reduces each variant only when enough stock remains. It writes one sale and item snapshots before committing. A failure rolls the entire transaction back and leaves the cart available for correction.

Transaction pattern from Sales::store

```
$db->transBegin();
$db->table('product_variants')
    ->set('stock_qty', 'stock_qty - ' . $quantity, false)
    ->where('id', $item['variant_id'])
    ->where('stock_qty >=', $quantity)->update();
if ($db->affectedRows() !== 1) {
    throw new RuntimeException('Stock changed.');
```

```
}
// Insert sale and item snapshots, then transCommit().
// On any error, transRollback().
```

A receipt reads the saved sale_items values, so later catalog edits do not change the recorded product name, option or unit price. This demo records sales; it does not process payment cards.

Customer checkout and website orders

A shopper can open /checkout directly from Cart. The form asks for a full name and email, with an optional phone number. Orders::store checks these details and the current cart on the server. It reserves stock with conditional updates and saves a pending web_orders record plus item snapshots in one transaction. Failed updates roll back all changes and keep the cart.

Order stock reservation pattern

```
$db->table('product_variants')
->set('stock_qty', 'stock_qty - ' . $quantity, false)
->where('id', $item['variant_id'])
->where('stock_qty >=', $quantity)->update();
if ($db->affectedRows() !== 1) {
    throw new RuntimeException('Stock changed.');
```

After a successful submission, the cart clears and the shopper sees a confirmation at an unguessable token URL. The confirmation is an order request, not a payment receipt. No payment or shipping is processed. Staff can view website orders at /orders, mark a pending order fulfilled, or cancel it. Cancellation returns reserved stock in the same transaction as the status update. The two assessment tables remain unchanged.

Interface and verification

The existing custom logo, cream and forest green palette, WebP images and hero art remain. Text and tap targets are larger. The desktop catalog uses three columns; small screens use two then one. Cards gain subtle depth, clearer specifications and visible stock. Keyboard focus and reduced-motion styles are retained and extended.

Observed checks

Area	Result
TA2 schema	Five customers and five users preserved with the exact supplied columns.
Catalog	Nine products and 29 stocked variants read from the database.
Account pages	Search, sort, matched counts and no-match state rendered.
Staff login	Generated password verified; wrong password and signed-out access rejected.
Cart and stock	Variant quantity and stock limits checked.
Sale	Sale, item snapshot and stock deduction checked inside a rollback-only verification transaction.
Failed sale	Insufficient stock left sales and stock unchanged.

The earlier direct PHP and database check passed 34 assertions for the TA2 account pages and in-person POS. Customer checkout and website order management were added afterward and are not covered by that count. Browser access was denied during preparation, so a visual browser pass remains for the user to perform. Word layout could not be rendered on this machine; the separate PDF was inspected page by page.

Manual browser walkthrough

Open Shop, filter a category and add a stocked option. Change its quantity in Cart and place a website order without signing in. Check the confirmation. Then sign in, open Orders and update an order status; cancel a separate sample order to check that stock returns. Search a customer, update a variant in Inventory, record a small fictional in-person sale, and inspect Sales. Narrow the window and check keyboard focus and table scrolling.

Maintenance and future repository

Location	Purpose
database/schema.sql; sample-data.sql	Exact TA2 tables and ten fictional account records
database/commerce-*.sql	Separate product, stock, credentials, orders and sales extension
app/Models/	Table mappings for accounts and POS records
app/Controllers/	Page flow, search, cart, orders, inventory, login and sales
app/Libraries/CartService.php	Session cart and stock validation
app/Filters/StaffOnly.php	Protection for staff routes
app/Views/; public/assets/	Escaped HTML views, product images and responsive CSS
tools/; Start-TA2.cmd	Safe local database and website startup

GitHub later

The TA2 project is independent of TFA1 Git history. For a new repository, commit source, composer.lock, SQL files and .env.example. The included .gitignore excludes .env, .local, vendor and runtime logs. The complete ZIP contains vendor for this machine; a fresh Git clone runs composer install. GitHub Pages cannot execute this PHP site. Public deployment needs a PHP host and a separate security review for HTTPS, credentials, data access and backups.

Source and scope

The core TA2 requirements come from the assessment outline supplied in the conversation and the Module 2 skill. The user requested additional inventory, cart, sales and staff features, then clarified that website shoppers must place orders without staff sign-in. The website order tables are separate, so the required TA2 schema remains intact. Full working code is in the project files cited throughout this guide.